```
$ python3 day01.py
```

# Python Day01

## # 變數・型別・輸入輸出

```
>>> print("從今天開始，學會跟程式對話")
從今天開始，學會跟程式對話
```

LucasHsu.dev — 2026　　約 45 分鐘



$ cat agenda.md

# 今日大綱

1. Python Day01 — 變數・型別・輸入輸出
2. 資料型別
3. 一個容器，五種形狀
4. 型別的層級關係
5. 看起來像，但不是同一種
6. 變數
7. 變數宣告
8. 命名小提醒
9. 進階：同時賦值
10. Output
11. 從「印出」到「組合」
12. 控制輸出格式
13. f-string 格式化技巧

PART 01

# 資料型別

程式眼中的世界，只有幾種「形狀」

# 一個容器，五種形狀

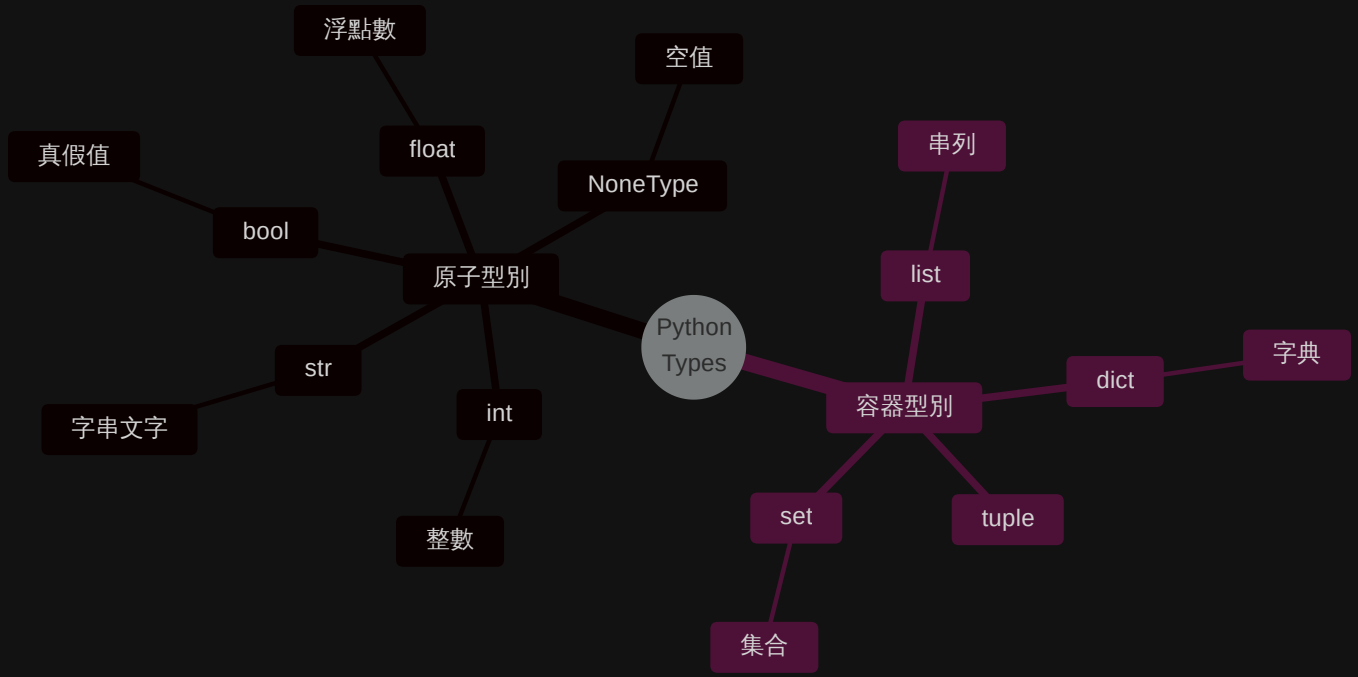
Python 把資料分成幾種基本「形狀」(type)，每種形狀能做的事情不一樣：

| 10 | 12.3 | 'hi' | True | None |
| --- | --- | --- | --- | --- |
| int<br>整數 | float<br>浮點數 | str<br>字串 | bool<br>布林 | NoneType<br>空值 |

```
>>> type(10) # <class 'int'>
>>> type('hi') # <class 'str'>
```

# 型別的層級關係

資料型別可以分成兩大類：「原子」和「容器」——理解這個分類，後面的學習會更有方向：




🔑 今天只談**原子型別**（左半邊），容器型別是之後課程的主題

# 看起來像，但不是同一種

這是初學者最容易踩的坑——**看起來一樣的資料，形狀不同就不能混用**：

```
# 引號決定型別
'123' ~ 字串
123 ~ 整數
'123' == 123 → False
```

**`'123'`**

有引號包住 → **字串**

電話、身分證字號常用這個

**`123`**

沒有引號 → **數字**

可以拿來做加減乘除

```
>>> '123' == 123
False # 形狀不同，永遠不相等
```

🔑 **記住這個結論**：等一下講 `input()` 時會用到它

PART 02

# 變數

幫資料貼上一張可以重複使用的標籤

# 變數宣告

用 `=` 把一個值「貼標籤」存起來，之後就能用這個名字取代它：

```python
int_val = 10
float_val = 12.3
str_val = 'hello python'
boolean_val = True
nv = None

print(int_val, float_val, str_val, boolean_val)
print(type(int_val))   # <class 'int'>
print(type(str_val))   # <class 'str'>
print(nv)              # None
```

💡 變數名字是你取的，但型別是 Python 自己根據「值長什麼樣子」判斷出來的

🤔 試試看在下方編輯器中修改變數的值，看看輸出怎麼變：

```python
# ✏️ 試著修改這些值！
int_val = 10
float_val = 12.3
str_val = 'hello python'
boolean_val = True
```

# 命名小提醒

## ✅ 好的命名

```
age = 18
user_name = 'Lucas'
total_price = 299
```

語意清楚，看名字就懂用途

## ❌ 避免這樣

```
a = 18
x1 = 'Lucas'
data = 299
```

過幾天連自己都看不懂

```
# 慣例：變數名用小寫 + 底線
>>> user_name ← snake_case（Python 標準）
>>> userName ← camelCase（Python 不常用）
```

⚠️ 不能以數字開頭、不能包含特殊符號（除了 `_`）、不能是保留字（如 `if`, `for`, `while`）

# 進階：同時賦值

一次把多個變數「同時」貼上標籤：

同時賦值 交換變數 相同值

```
a, b = 10, 20
print(a)  # 10
print(b)  # 20
```

✨ **變數交換**是 Python 的特色語法——其他語言通常需要一個「暫存變數」才能做到

PART 03

# Output

用 print() 把資料說出來

# 從「印出」到「組合」

```
print('hi')
print('hi')       # 預設換行，所以 hi 印兩行
```

輸出結果：

```
hi
hi
```

但只會印「固定內容」是不夠的——來看看變數怎麼跟文字結合：

```
# 😵 用逗號 — 變亂了
name = 'Lucas'
age = 18
print('Hello,', name, 'you are', age, 'years old')
```

```
# 😊 用 f-string — 乾淨多了
name = 'Lucas'
age = 18
print(f'Hello, {name}! You are {age} years old.')
```

```
# 🤯 f-string 裡還能放運算式
name = 'Lucas'
age = 18
print(f'Hello, {name}! Next year you will be {age + 1
```

# 控制輸出格式

`print()` 的完整面貌——它其實有很多參數可以用：

預設換行 | 取消換行 | 自訂分隔 | 全部自訂

```
print('hi')
print('hi')
# 輸出:
# hi
# hi
```

🔑 關鍵參數：`end` — 控制結尾（預設 `'\n'`）· `sep` — 控制分隔（預設 `' '`）

# f-string 格式化技巧

f-string 不只能放變數，還能控制「怎麼顯示」：

```python
# 基本用法
print(f'{變數}')

# 數字格式化：控制小數位數
pi = 3.14159265
print(f'π ≈ {pi:.2f}')   # π ≈ 3.14
print(f'π ≈ {pi:.4f}')   # π ≈ 3.1416

# 數字補零
print(f'{42:05d}')        # 00042
```

### 格式語法

`{值:格式}`

`:.2f` — 兩位小數

`:05d` — 五碼補零

### ✏️ 互動練習

修改下方數字，看格式變化：

```python
pi = 3.14159265
print(f'π ≈ {pi:.2f}')
print(f'π ≈ {pi:.4f}')
print(f'{42:05d}')
```

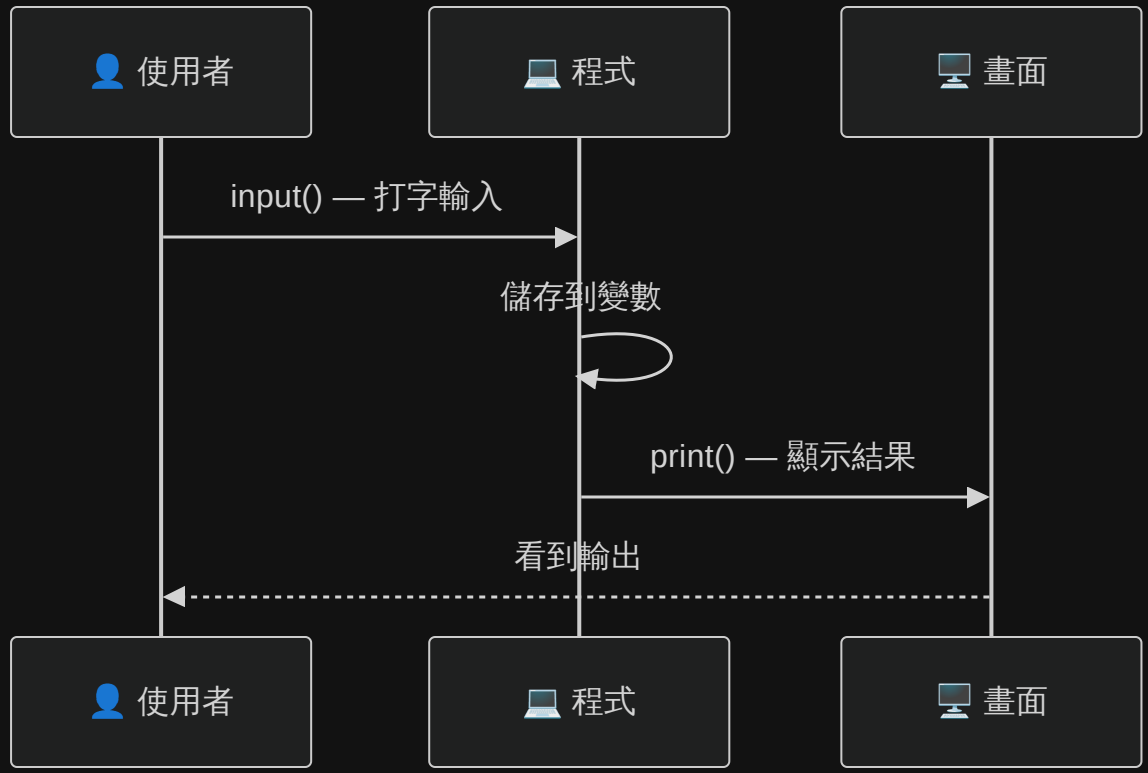
PART 04

# Input

讓程式可以「聽」使用者說話

# 資料的流向

程式如何跟使用者互動？資料從鍵盤流進變數，再從變數流到畫面：



# 單行輸入

用 `input()` 接收使用者從鍵盤打的內容：

```
n = input()
nn = input('請輸入一些字：')
```

⚠️ `input()` 不管你輸入什麼，回傳的**永遠是字串**
（還記得 `'123' != 123` 嗎？）

# 範例

```
# ✏️ 點擊執行後在下方輸入
name = input('你的名字？ ')
print(f'Hello, {name}!')

# input() 回傳的是字串！
age_str = input('幾歲？ ')
print(type(age_str))  # <class 'str'>

age = int(age_str)      # 轉成整數
print(f'明年你就 {age + 1} 歲了')
```

# 多行輸入

當你需要處理多筆資料時，有三種常見做法：

sys.stdin | while + try | 固定行數

```python
import sys

for line in sys.stdin:
    line = line.strip()
    print(line)
```

📂 **sys.stdin**
最常用
適合競賽程式

🔄 **while+try**
明確控制
適合互動輸入

📋 **固定行數**
最直觀
適合已知數量

# 檔案輸入：用 < 餵資料給程式

建立 `a.py`：

```
a = input()
b = input()
print(a, b)
```

建立 `in.txt`：

```
Everything will get better
12345
```

**終端機執行：**

```
python a.py < in.txt
```

**輸出結果：**

```
Everything will get better 12345
```

💡 `<` 把檔案內容導向成標準輸入，等於自動幫你打字

⚠️ 這是競賽程式和自動化測試的常見做法，習慣它之後就不用每次都手打輸入

# 中文字元的編碼陷阱

如果 `in.txt` 裡有中文字元，預設編碼可能會出問題：

```python
import sys

# 明確指定 UTF-8 編碼
sys.stdin.reconfigure(encoding='utf-8')

for line in sys.stdin:
    line = line.strip()
    print(line)
```

輸出結果存成檔案：

```
python a.py < in.txt > out.txt
```

⚠️ 沒設定 UTF-8 編碼時，中文字元在某些系統（尤其 Windows）上可能會變亂碼

PART 05

# 數學運算子

讓數字動起來

# 運算子總覽

| 運算 | 程式碼 | 說明 | 範例結果 |
| --- | --- | --- | --- |
| 加法 | `n + 2` | Addition | `n=10 → 12` |
| 減法 | `n - 2` | Subtraction | `n=10 → 8` |
| 乘法 | `n * 2` | Multiplication | `n=10 → 20` |
| 除法 | `n / 2` | Division（一律回傳 float） | `n=10 → 5.0` |
| 整數除法 | `n // 2` | 無條件捨去取整數 | `n=10 → 5` |
| 次方 | `n ** 6` | Power | `n=10 → 1000000` |
| 取餘數 | `n % 6` | Modulo（常用來判斷奇偶、倍數） | `n=10 → 4` |

🧠 最值得記下來的三個：`/`（小數除法）· `//`（整數除法）· `%`（取餘數）

# 實際範例

```
n = 10
```

`n + 2` → **12**

`n - 2` → **8**

`n * 2` → **20**

`n / 2` → **5.0** ← float！

`n // 2` → **5**

`n ** 6` → **1000000**

`n % 6` → **4** ← 10 ÷ 6 的餘數

💡 `/` 一律回傳 `float`，`//` 回傳無條件捨去的整數

# 算術指定運算子

把運算和指定合併成一行：

一般寫法 簡寫 全部運算子

```
x = 10
x = x + 5   # x 變成 15
x = x * 2   # x 變成 30
print(x)
```

⚡ `x += 1` 是最常用的寫法——不管是計數器、累加值還是迴圈控制都會用到它

PART 06

# 比較與條件判斷

讓程式自己做選擇

# 布林值與比較運算子

布林是只有兩個值的型別：

- ✓ `True`（真）
- ✗ `False`（假）

| 運算子 | 效果 |
| --- | --- |
| `<` | 小於 |
| `<=` | 小於等於 |
| `>` | 大於 |
| `>=` | 大於等於 |
| `==` | 等於 |
| `!=` | 不等於 |

## 範例

```
10 <= 60      # True
123 == 123    # True
20 == 40      # False
'123' == 123  # False  ⚠️
```

⚠️ 字串 `'123'` 與整數 `123` 型別不同，比較結果永遠是 `False`

# 邏輯運算子（Logical Operators）

把多個條件組合起來：

| 運算子 | 意思 |
| --- | --- |
| `and` | 兩個條件都要成立 |
| `or` | 其中一個成立即可 |
| `not` | 反轉結果 |

```
age = 18
has_id = True

age >= 18 and has_id     # True
age < 18 or has_id       # True
not has_id               # False
```

🧠 `and` = 全部要過 · `or` = 有一個過就好 · `not` = 翻轉

# Truthy / Falsy（重要觀念）

在 Python 中，不只有 True / False 才能當條件：

```
bool(0)        # False
bool(1)        # True
bool("")       # False
bool("hi")     # True
bool(None)     # False
bool([])       # False
```

## Falsy（視為 False）

- `0`
- `""`
- `None`
- `[]`
- `{}`

## Truthy（視為 True）

- 非 0 數字
- 非空字串
- 非空容器

# if 條件判斷

讓程式「做選擇」：

```
age = 18

if age >= 18:
    print("可以進入")
```

⚠️ Python 用「縮排」決定程式區塊，不是大括號

# if / elif / else

多條件分支：

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("F")
```

💡 條件會「由上往下」檢查，第一個成立就停止

# 實戰：登入判斷

```python
username = input("帳號：")
password = input("密碼：")

if username == "admin" and password == "1234":
    print("登入成功")
else:
    print("登入失敗")
```

🔑 同時使用：input + 字串 + and 條件判斷

# 小練習（一定要做）

```python
n = int(input("請輸入一個數字："))

if n % 2 == 0:
    print("偶數")
else:
    print("奇數")
```

挑戰：改成判斷「是否為 3 的倍數」

# 今日重點回顧

- Python 的基本型別：int / float / str / bool / None
- `input()` 永遠回傳 **字串**
- `print()` 可以用 f-string 組字串
- `/`、`//`、`%` 差異
- 比較運算 + 邏輯運算
- `if / elif / else` 控制流程

# 下一步

你已經可以：

- 接收輸入
- 做計算
- 做條件判斷
- 輸出結果

👉 下一堂：**迴圈（Loop）與重複執行**